

ESERCIZIO PRATICO

Backend REST API con Node.js & Express

Sistema di Gestione Salone di Parrucchiere

Informazioni esercizio

Livello	Intermedio
Prerequisiti	JavaScript ES6+, concetti REST, SQL (funzioni di aggregazione e stringa)
Tecnologie	Node.js 20+, Express 4, Sequelize 6, MariaDB
Endpoint richiesti	6 endpoint totali, tutti di sola lettura (GET)
Consegna	Codice sorgente, script SQL (creazione tabelle + dati), collection Postman

1. Obiettivi dell'esercizio

Lo studente dovrà progettare e implementare un backend REST di sola lettura per la gestione di un salone di parrucchiere, usando Node.js con Express e Sequelize come ORM. A differenza degli esercizi precedenti, qui non sono richieste operazioni di scrittura (POST, PUT, DELETE): l'obiettivo è concentrarsi sulla scrittura di query di lettura evolute, che sfruttano funzioni SQL di concatenazione, conversione e aggregazione direttamente nella SELECT.

Al termine dell'esercizio lo studente sarà in grado di:

- Definire modelli Sequelize con una relazione multi-a-molti tramite tabella associativa con attributi propri
- Utilizzare `sequelize.fn` e `sequelize.literal` per richiamare funzioni SQL native (CONCAT, DATE_FORMAT, TIMESTAMPDIFF, ROUND)
- Restituire campi calcolati e formattati nelle risposte JSON, senza modificarli lato JavaScript
- Scrivere query di aggregazione con COUNT, SUM, AVG e GROUP BY
- Effettuare ricerche testuali case-insensitive con l'operatore LIKE
- Produrre script SQL per la creazione delle tabelle e il popolamento dei dati di test

2. Struttura del database

Il database MariaDB deve contenere due tabelle principali collegate da una relazione molti-a-molti, gestita tramite una tabella associativa che, a differenza degli esercizi precedenti, contiene anche attributi propri (data dell'appuntamento e valutazione del cliente).

2.1 Tabella principale — Cliente

Rappresenta un cliente iscritto al salone.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	NOT NULL	Nome del cliente
cognome	STRING	NOT NULL	Cognome del cliente
email	STRING	UNIQUE, NOT NULL	Indirizzo email del cliente
telefono	STRING	opzionale	Numero di telefono
dataNascita	DATEONLY	NOT NULL	Data di nascita, usata per calcolare l'età
dataIscrizione	DATE	default NOW	Data e ora di iscrizione al salone

2.2 Tabella secondaria — Servizio (relazione Molti-a-Molti con Cliente)

Rappresenta un servizio offerto dal salone (es. "Taglio", "Colore", "Piega"). Un cliente può aver prenotato molti servizi nel tempo; uno stesso servizio può essere stato prenotato da molti clienti.

In Sequelize: `Cliente.belongsToMany(Servizio, { through: 'Prenotazione' })` e `Servizio.belongsToMany(Cliente, { through: 'Prenotazione' })`.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	UNIQUE, NOT NULL	Es. "Taglio", "Colore", "Piega", "Shampoo"
descrizione	TEXT	opzionale	Descrizione del servizio
prezzo	FLOAT	NOT NULL	Prezzo del servizio in euro
durataMinuti	INTEGER	NOT NULL	Durata del servizio espressa in minuti

2.3 Tabella associativa — Prenotazione

A differenza della semplice tabella pivot vista negli esercizi precedenti, in questo caso la tabella associativa contiene anche dati propri dell'appuntamento, e non solo le due chiavi esterne.

Campo	Tipo	Vincoli	Note
Clientid	INTEGER	PK composta, FK → Cliente	Chiave esterna verso Cliente
Servizioid	INTEGER	PK composta, FK → Servizio	Chiave esterna verso Servizio

dataAppuntamento	DATE	NOT NULL	Data e ora in cui è stato erogato il servizio
valutazione	INTEGER	opzionale, 1-5	Valutazione data dal cliente al servizio ricevuto

Nota sul diagramma ER

Prima di scrivere codice, disegnare il diagramma ER delle due tabelle principali e della tabella associativa Prenotazione, evidenziando che quest'ultima ha una chiave primaria composta (Clienteld + Serviziold) e due colonne aggiuntive che non sono semplici chiavi esterne.

3. Tecnologie e struttura del progetto

3.1 Stack tecnologico richiesto

- Node.js 20 LTS o superiore
- Express 4 — framework web per la creazione di REST API
- Sequelize 6 — ORM per Node.js compatibile con MariaDB, MySQL, PostgreSQL
- mariadb — driver nativo Node.js per MariaDB (alternativa: mysql2)
- dotenv — gestione delle variabili di ambiente per le credenziali del database

3.2 Struttura consigliata del progetto

```
salone-parrucchiere/
├── app.js                # entry point Express
├── config/
│   └── database.js      # connessione Sequelize + MariaDB
├── models/
│   ├── index.js         # carica modelli e associazioni
│   ├── Cliente.js       # modello Sequelize Cliente
│   └── Servizio.js      # modello Sequelize Servizio
├── routes/
│   ├── clienti.js       # router endpoint clienti
│   ├── servizi.js       # router endpoint servizi
│   └── statistiche.js    # router endpoint statistiche
├── sql/
│   ├── create_tables.sql # script di creazione tabelle
│   └── seed_data.sql     # script di popolamento dati
├── postman/
│   └── salone_parrucchiere.json # collection Postman
├── .env                  # credenziali DB (non committare!)
├── package.json
└── README.md
```

4. Elenco degli endpoint richiesti

Il progetto deve esporre esattamente 6 endpoint, tutti di tipo GET. Ogni endpoint deve restituire codice HTTP 200 in caso di successo, 404 se la risorsa richiesta non esiste, e rispondere sempre con JSON. Ogni endpoint richiede l'utilizzo di almeno una funzione SQL di concatenazione, conversione o aggregazione all'interno della query.

#	Metodo	URL	Descrizione	Tag
1	GET	/api/clienti	Lista clienti con nome completo concatenato	Cliente
2	GET	/api/clienti/:id	Dettaglio cliente con età calcolata e servizi svolti	Cliente
3	GET	/api/clienti/ricerca	Ricerca clienti per nome o cognome (?q=...)	Cliente
4	GET	/api/servizi	Lista servizi con durata e prezzo formattati	Servizio
5	GET	/api/servizi/:id	Dettaglio servizio con valutazione media dei clienti	Servizio
6	GET	/api/statistiche/servizi-popolari	Classifica servizi per numero di prenotazioni e incasso	Statistiche

5. Specifiche dettagliate degli endpoint

Endpoint 1 — GET /api/clienti

Deve restituire l'elenco dei clienti includendo un campo calcolato nomeCompleto ottenuto concatenando nome e cognome lato database, non lato JavaScript. Usare `sequelize.fn` e `sequelize.col`:

```
const clienti = await Cliente.findAll({
  attributes: [
    'id', 'email', 'telefono',
    [sequelize.fn('CONCAT', sequelize.col('nome'), ' ',
      sequelize.col('cognome')), 'nomeCompleto'],
    [sequelize.fn('DATE_FORMAT', sequelize.col('dataIscrizione'),
      '%d/%m/%Y'), 'dataIscrizioneFormattata']
  ]
});
```

Endpoint 2 — GET /api/clienti/:id

Deve restituire il dettaglio del cliente con un campo calcolato eta, ottenuto dalla differenza in anni tra la data odierna e dataNascita tramite la funzione SQL `TIMESTAMPDIFF`, e l'elenco dei servizi svolti (tramite `include` su Servizio, includendo anche gli attributi della tabella Prenotazione: `dataAppuntamento` e `valutazione`).

```
attributes: {
  include: [
    [sequelize.fn('TIMESTAMPDIFF', sequelize.literal('YEAR'),
```

```
      sequelize.col('dataNascita'), sequelize.fn('NOW')), 'eta']
    ]
  }
}
```

Se il cliente non esiste, restituire HTTP 404 con il messaggio: "Cliente non trovato."

Endpoint 3 — GET /api/clienti/ricerca?q=...

Deve effettuare una ricerca case-insensitive su nome e cognome usando l'operatore Op.like, restituendo comunque il campo nomeCompleto concatenato come nell'endpoint 1. Esempio: /api/clienti/ricerca?q=ross deve trovare sia "Rossi" che "Rossetti". Se il parametro q non viene fornito, restituire HTTP 400 con il messaggio: "Parametro di ricerca 'q' obbligatorio."

Endpoint 4 — GET /api/servizi

Deve restituire l'elenco dei servizi con due campi calcolati: durataFormattata (durata convertita da minuti in formato "Xh Ymin" tramite funzioni SQL di divisione intera e modulo) e prezzoFormattato (prezzo con due decimali concatenato al simbolo dell'euro tramite CONCAT).

```
[sequelize.literal("CONCAT(FLOOR(durataMinuti/60), 'h ',
MOD(durataMinuti,60), 'min')"), 'durataFormattata'],
[sequelize.literal("CONCAT(FORMAT(prezzo, 2), ' €')"), 'prezzoFormattato']
```

Endpoint 5 — GET /api/servizi/:id

Deve restituire il dettaglio del servizio con il numero totale di prenotazioni ricevute (COUNT) e la valutazione media arrotondata a un decimale (AVG + ROUND), oltre all'elenco dei clienti che lo hanno prenotato. Se il servizio non esiste, restituire HTTP 404 con il messaggio: "Servizio non trovato."

Endpoint 6 — GET /api/statistiche/servizi-popolari

Deve restituire una classifica dei servizi ordinata per numero di prenotazioni decrescente, con i campi: nome del servizio, numeroPrenotazioni (COUNT), incassoTotale (SUM del prezzo per ciascuna prenotazione) e valutazioneMedia (AVG arrotondata). Richiede una query con GROUP BY sul servizio e ORDER BY sul conteggio.

6. Definizione dei modelli e delle associazioni

6.1 Associazioni con attributi sulla tabella pivot

Poiché la tabella Prenotazione contiene colonne proprie, è necessario definirla come modello Sequelize a sé stante e usarla come through esplicito, anziché lasciare che Sequelize la generi automaticamente:

```
Cliente.belongsToMany(Servizio, { through: Prenotazione });
Servizio.belongsToMany(Cliente, { through: Prenotazione });
```

In questo modo, includendo Servizio in una query su Cliente, sarà possibile accedere anche ai campi dataAppuntamento e valutazione tramite l'oggetto Prenotazione annidato in ogni risultato.

6.2 Include con attributi della tabella pivot

```
const cliente = await Cliente.findByPk(id, {
  include: [{
    model: Servizio,
    through: { attributes: ['dataAppuntamento', 'valutazione'] }
  }]
});
```

7. Script SQL richiesti

Nella cartella sql/ devono essere presenti due file SQL eseguibili direttamente su MariaDB.

7.1 create_tables.sql

Deve contenere le istruzioni CREATE TABLE per le tre tabelle (Clienti, Servizi, Prenotazioni), con i vincoli di chiave primaria (composta per Prenotazioni), chiave esterna e NOT NULL. Usare IF NOT EXISTS per rendere lo script rieseguibile.

7.2 seed_data.sql

Deve contenere le istruzioni INSERT INTO per popolare il database con dati di test realistici. Sono richiesti almeno:

- 10 clienti con date di nascita ed email diverse
- 6 servizi con durate e prezzi variati
- almeno 20 prenotazioni distribuite tra clienti e servizi, con date e valutazioni diverse, in modo che le query di aggregazione dell'endpoint 6 restituiscano risultati significativi

8. Requisiti di consegna

Il materiale da consegnare è composto da tre elementi obbligatori:

1. Codice sorgente Node.js — Cartella del progetto completa e funzionante, con file package.json contenente tutte le dipendenze e uno script "start" per avviare il server. Il codice deve essere privo di errori e rispettare la struttura delle cartelle indicata al punto 3.2.
2. Script SQL — I due file create_tables.sql e seed_data.sql nella cartella sql/. Gli script devono essere eseguibili su un server MariaDB pulito senza alcuna modifica manuale.
3. Collection Postman — File .json importabile in Postman con tutti e 6 gli endpoint configurati, organizzati in cartelle (Clienti, Servizi, Statistiche) che rispecchino i tag degli endpoint. Includere, dove utile, esempi di query string (es. ?q=ross).

Avvertenza

L'uso di soluzioni generate automaticamente da strumenti di intelligenza artificiale senza comprensione del codice prodotto è considerato disonestà accademica. Il docente si riserva di richiedere la spiegazione di qualsiasi parte del materiale consegnato.